MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY — FPTU QUY NHON AI CAMPUS

FU CAHATA · FACULTY OF INFORMATION TECHNOLOGY — SOFTWARE ENGINEERING

OMNICAST MOBILE | NATIVE FLUTTER CLIENT

FINAL PROJECT DELIVERABLE REPORT

COURSE: PRM393 — MOBILE APPLICATIONS DEVELOPMENT

OMNICAST MOBILE: ENTERPRISE BROADCAST & INTERACTIVE STREAMING CLIENT

Broadcast. Connect. Discover.

4-tier Clean Architecture and BLoC; LiveStream and Mobile Live Studio WebRTC WHIP; offline-first SQLite; OmniPlayer

Mentor / Supervisor:	**Mr. Huỳnh Vang** (Software Engineering Dept. — FPTU)
Campus & Team Note:	**FPTU Quy Nhon AI Campus — FU Cahata**
Academic Semester:	Fall 2026 (FA26)
Project Team Members (4):	**1. Nguyễn Bình An** (Team Leader / Backend & Security) **2. Trần Thanh Sơn** — MSSV: `QE170080` (gRPC Audit & Database) **3. Nguyễn Vĩnh Phú** — MSSV: `QE180136` (Mobile Flutter Lead) **4. Đồng Hoàng Nguyên** (Full-Stack Web, Live & AI)
GitHub Repository:	https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester
Mobile Release (APK):	https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester/releases/tag/v1.2.0
Scale & Verification:	**53,922 Dart LOC | 31 screens | 91 tests in tree; CI run confirms 77/77**

FPTU QUY NHON AI CAMPUS — FU CAHATA · OCTOBER 2026

Cross-Platform Mobile Application Development: The OmniCast Mobile Broadcast and Digital Content Delivery Client

Nguyễn Bình An, **Trần Thanh Sơn** (QE170080), **Nguyễn Vĩnh Phú** (QE180136), **Đồng Hoàng Nguyên**

Mentor / Supervisor: **Mr. Huỳnh Vang**
FPTU Quy Nhon AI Campus — FU Cahata, Faculty of Information Technology, FPT University
Integrated PRM393 mobile application report

***Abstract*— OmniCast Mobile is an enterprise cross-platform digital broadcast, LiveStream and multimedia streaming client engineered at FPTU Quy Nhon AI Campus — FU Cahata for the PRM393 course under the supervision of Mentor Mr. Huỳnh Vang utilizing Flutter 3.x and Dart. The client adheres to a 4-tier Clean Architecture coupled with reactive stream-based BLoC state management spanning 12 business modules and 31 screens. Four core technological contributions are presented: (1) the multi-engine `OmniPlayer` uniting ultra-low-latency WebRTC/HLS streams via MediaMTX, direct Google Drive stream resolution, dynamic Wi-Fi LAN host discovery (`LanDiscoveryService`), and bi-directional real-time interactive live rooms (`LiveWatchScreen`: threaded Live Chat, 0ms haptic Floating Reactions, 6-tier Super Chat Donations, pinned broadcaster banners and zero-downtime program switching); (2) a 4-tab `LiveStudioScreen` and professional field-reporter camera viewfinder (`MobileStudioRecorderScreen`) featuring Push-to-Live WebRTC WHIP publishing, SMPTE timecode, Stereo VU audio meters and an auto-scrolling Teleprompter HUD; (3) an offline-first engine driven by local SQLite storage featuring an asynchronous mutation replay queue (`pending_ops`) and exact 15-minute advance local alarms; and (4) a declarative GoRouter topology governing 31 screens with strict session-guarded route protection. The software The test tree contains 91 tests. GitHub Actions run 40630318005 executed 77 tests in workflow scope, and all 77 passed.**

***Index Terms*—Flutter, Dart, BLoC, LiveStream, Live Studio, WebRTC WHIP, SQLite, offline-first, MediaMTX, HLS, GoRouter, FPTU Quy Nhon AI Campus, FU Cahata.**

I. Introduction

In the modern era of digital content delivery, mobile applications have become the primary touchpoint connecting viewers with broadcast media services. Unlike desktop environments with steady broadband connectivity and continuous power supplies, mobile devices present stringent technical challenges: highly fluctuating network latency, frequent network drops during roaming transitions (cell tower handovers or Wi-Fi to cellular switches), constrained memory footprints, and battery budget limits.

To comprehensively address these demands, this paper presents the architectural design and implementation of **OmniCast Mobile** — an enterprise cross-platform digital broadcast, LiveStream and multimedia streaming client developed at **FPTU Quy Nhon AI Campus — FU Cahata** for the course **PRM393 — Mobile Applications Development** under the academic mentorship of **Mr. Huỳnh Vang**. Built upon Flutter 3.x with the Dart language (53,922 Dart LOC), the client adopts a clean four-tier layered architecture coupled with the Business Logic Component (BLoC) state management pattern across 31 screens and 22 custom widgets.

A. Motivation and Engineering Challenges

The engineering of OmniCast Mobile resolves five pivotal technical challenges:

1) **Offline-First Resilience:** Viewers frequently consult electronic programme guides (EPG) and their personal watchlists in poor or nonexistent connectivity conditions. The application must store structured data in a local SQLite database, queue pending mutations while offline, and seamlessly replay them to the central cloud upon reconnection.
2) **Low-Latency Adaptive Streaming and LAN Host Discovery:** The OmniCast broadcast pipeline emits ultra-low-latency WebRTC/HLS streams via MediaMTX, direct Google Drive streams, and external curated feeds. The embedded `OmniPlayer` must dynamically scan and discover local Wi-Fi MediaMTX hosts (`LanDiscoveryService`) and recover transparently during transient network drops.
3) **Bi-Directional Interactive LiveStream & Mobile Live Studio:** Viewers expect rich interactive rooms with lobby prompts, threaded live chat, 0ms haptic floating reactions, pinned host banners, and 6-tier Super Chat donations. Meanwhile, field reporters require a 4-tab `LiveStudioScreen` and hardware camera viewfinder (`MobileStudioRecorderScreen`) capable of real-time WebRTC WHIP publishing.
4) **Distributed State Management and Session Security:** Navigating 31 screens powered by 12 BLoC modules requires predictable unidirectional data flows, paired with hardware-backed secure storage (Keystore/Keychain) for transparent JWT token rotation.

Table I
THE 4 PROJECT TEAM MEMBERS OF OMNICAST (FPTU QUY NHON AI CAMPUS — FU CAHATA, MENTOR: MR. HUỲNH VANG).

Full Name	Role / ID	Primary Engineering Ownership
Nguyễn Bình An (`TomOutfit`)	Team Leader	NestJS 19-module core architecture, JWT & 4-role RBAC security, EPG conflict detection SQL, Direct Stream Engine, CI/CD & Release
Trần Thanh Sơn	`QE170080`	Standalone gRPC Audit Logger microservice (`audit_logger.proto`), Supabase PostgreSQL (21 Prisma models), Telemetry & Seeding
Nguyễn Vĩnh Phú	`QE180136`	Mobile Flutter Lead (31 screens, 12 BLoCs), `OmniPlayer` engine, `MobileStudioRecorderScreen` (WebRTC WHIP), Offline SQLite
Đồng Hoàng Nguyên (`teng122`)	Full-Stack & AI	Next.js 15 Cinema Dark Web Portal (33 routes), Socket.IO `/live` gateway, 7-theme Live Room, Live Studio Console & AI Curator module

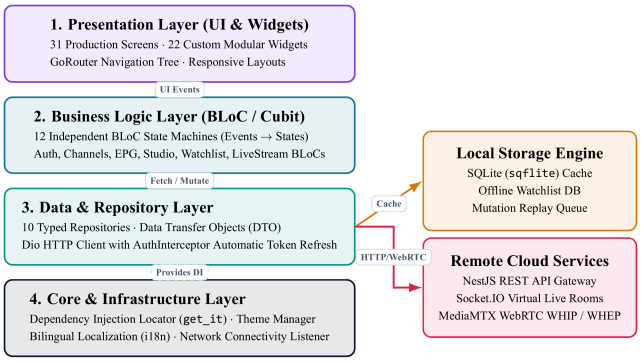


Figure 1. The 4-tier layered architecture of OmniCast Mobile: Presentation routed by GoRouter (31 screens, 22 widgets), Logic driven by 12 BLoC/Cubits, and Data communicating concurrently with NestJS REST APIs, Socket.IO `/live` and local SQLite storage.

B. Project Team Organization and Role Allocation

The project was executed by a four-member student engineering team at **FPTU Quy Nhon AI Campus — FU Cahata** under the supervision of Mentor **Mr. Huỳnh Vang**, as detailed in Table I.

C. Principal Contributions

This paper details five principal technical contributions:

- A 4-tier Clean Architecture (Presentation, Logic/BLoC, Data, Core) orchestrated through `get_it` service locator.
- A complete mobile live broadcast ecosystem comprising `OmniPlayer`, `LiveWatchScreen`, the 4-tab `LiveStudioScreen` and the `MobileStudioRecorderScreen` field viewfinder (WebRTC WHIP, SMPTE timecode, Stereo VU meter, Teleprompter HUD).
- A stream-based reactive state architecture employing custom BLoC concurrency transformers such as `droppable` and `restartable`.
- An offline-first SQLite synchronization engine featuring an asynchronous mutation queue (`pending_ops`) and conflict resolution heuristics.
- The test tree contains 91 tests; CI run 40630318005 executed 77 tests in workflow scope and all 77 passed.

II. MOBILE SYSTEM ARCHITECTURE

OmniCast Mobile strictly adheres to the Separation of Concerns and Clean Architecture principles adapted for the Flutter framework. The codebase is organized into four distinct layers: Presentation, Logic (BLoC), Data, and Core, ensuring high testability, modular reusability, and long-term maintainability.

A. Layered Organization

The four architectural tiers are defined as follows:

1) **Presentation Layer:** Encompasses 31 screens and 22 reusable widgets. This tier is completely free of business computations, focusing solely on state rendering and event dispatching.
2) **Logic Layer (BLoC / Cubit):** Hosts 12 state-management modules. BLoCs accept incoming events from UI widgets, invoke domain repositories, and emit reactive state streams.
3) **Data Layer:** Encapsulates 10 Repositories, Data Sources, and 10 Data Transfer Objects (DTOs). It abstracts data origins, transparently querying SQLite or fetching remotely via Dio.
4) **Core Layer:** Supplies cross-cutting concerns: `GoRouter` navigation definitions, `get_it` dependency injection, `LanDiscoveryService` Wi-Fi subnet scanner, `WebRTCPublisherService` WHIP streamer, UI theme styling, i18n localization, and secure token storage.

B. Dependency Injection with GetIt

All Repositories, Data Sources, Network Clients, and BLoC instances have their lifecycles managed via the `get_it` service locator. In `injection_container.dart`, registrations are strictly partitioned between Lazy Singletons (for stateless services, DB helpers, and repositories) and Factories (for transient BLoC instances tied to screen lifecycles):

```dart
final sl = GetIt.instance;

Future<void> init() async {
  // BLoCs (Factory)
  sl.registerFactory(() => AuthBloc(sl()));
  sl.registerFactory(() => WatchlistBloc(sl()));
  sl.registerFactory(() => EpgBloc(sl()));

  // Repositories (Lazy Singleton)
  sl.registerLazySingleton<WatchlistRepository>(
    () => WatchlistRepositoryImpl(
      remoteDataSource: sl(),
      localDataSource: sl(),
    ),
  );

  // Core & Drivers
  sl.registerLazySingleton(() => DioClient(sl()));
  sl.registerLazySingleton(() => DatabaseHelper());
```

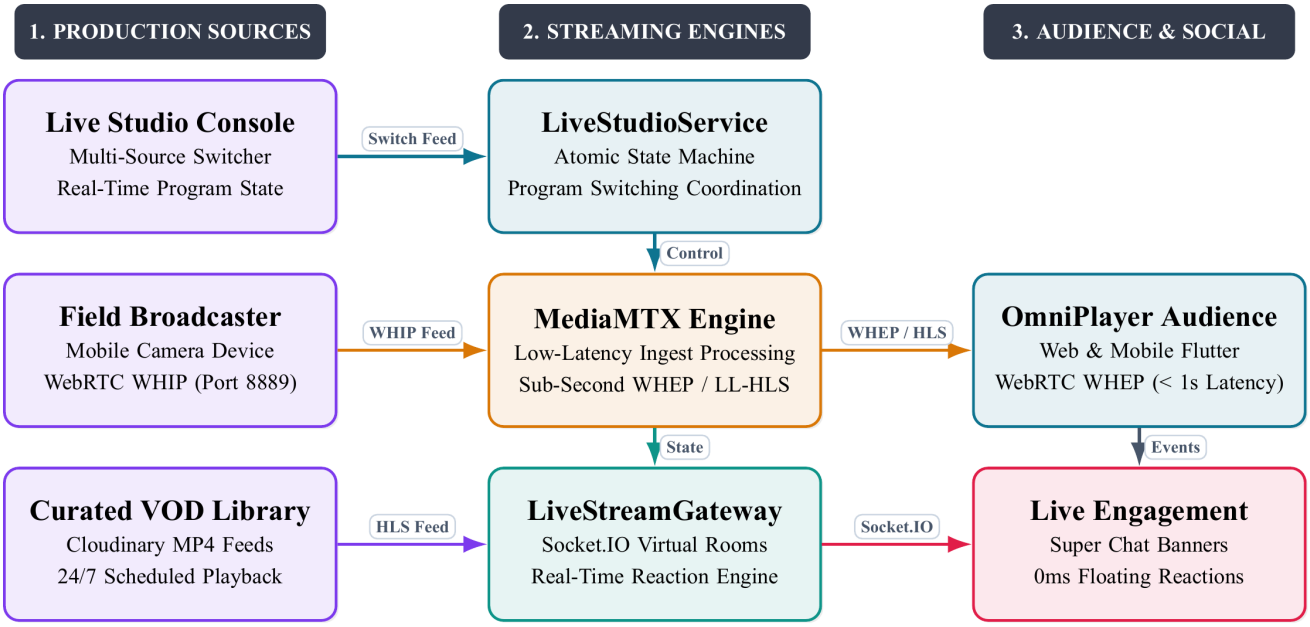


Figure 2. End-to-end LiveStream and Live Studio architecture bridging the Flutter mobile client, Next.js 15 web portal, MediaMTX (WHIP/WHEP/HLS) and the Socket.IO `/live` gateway.

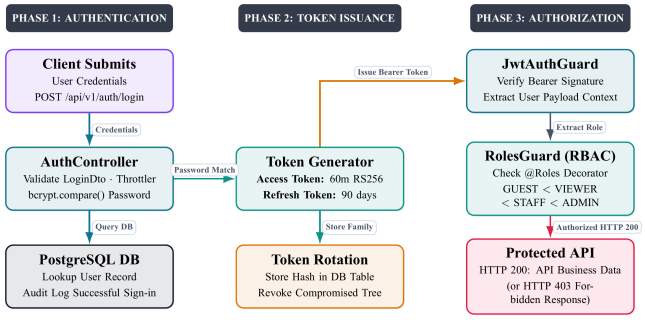


Figure 3. Layered JWT authentication and 4-tier RBAC access control flow connecting the Flutter client with the NestJS API.

```
}
```

Listing 1. Dependency injection configuration in `injection_container.dart`.

C. Networking and JWT Token Rotation

Remote HTTP communications are powered by `Dio` enriched with a custom `AuthInterceptor`. Upon intercepting an `HTTP 401 Unauthorized` response, the interceptor transparently initiates a background token refresh using the refresh token persisted in `flutter_secure_storage`:

1) Locks the active request queue to prevent cascading 401 errors.
2) Dispatches a refresh request to `/api/v1/auth/refresh` with the existing refresh token.
3) Persists the refreshed token pair and replays pending requests with the new Bearer token.
4) If the refresh token itself is invalid or revoked, safely clears the session and redirects the user to the Login screen.

III. State Management with BLoC

OmniCast Mobile adopts the Business Logic Component (BLoC) architectural pattern as its core state management strategy. BLoC leverages reactive Dart Streams to enforce strict unidirectional data flow: the UI dispatches an `Event`, the BLoC processes asynchronous business logic, and emits an immutable `State` reflecting the outcome.

A. Inventory of the 12 BLoC / Cubit Modules

The application's business logic is modularized into 12 dedicated units situated under `lib/logic/`:

1) **AuthBloc:** Manages user authentication lifecycle, login, registration, logout, and silent token rotation.
2) **WatchlistBloc:** Coordinates personal watchlist items, offline SQLite cache syncing, and local notification scheduling.
3) **EpgBloc:** Fetches and filters electronic programme guides per channel and date, consuming 24/7 seeded filler data.
4) **ChannelBloc:** Maintains broadcast channel catalogue, channel subscriptions, and category filters.
5) **LiveBloc:** Governs live broadcast playback, dynamic MediaMTX stream URL resolution, and player state.

6) **AiCuratorBloc:** Interfaces with the AI curator service, validating broadcasting compliance and generating editorial suggestions.
7) **ProgramsBloc:** Handles program catalogues, episode breakdowns, and production tags.
8) **RecordingsBloc:** Queries published on-demand broadcast recordings (VOD).
9) **SearchBloc:** Executes cross-entity searches across channels, events, and recordings with input debouncing.
10) **NotificationsBloc:** Handles in-app alerts and synchronizes Firebase Cloud Messaging push payloads.
11) **NewsletterBloc:** Manages channel newsletter subscriptions.
12) **StreamsBloc:** Aggregates currently active broadcast transmissions across all channels.

B. Concurrency Transformers

To manage high-frequency user interactions, OmniCast Mobile incorporates the `bloc_concurrency` package, enforcing custom event transformers:

- **droppable:** Applied to mutations like watchlist toggling or live donation dispatch. If an operation is already in-flight, subsequent duplicate user taps are safely dropped to prevent idempotent violations.
- **restartable:** Applied to `SearchBloc`. When a new query is typed, previous pending network requests are immediately cancelled, giving precedence to the latest user input.

C. Eliminating Redundant Rebuilds via Equatable

All state representations extend the `Equatable` class. Whenever a BLoC emits a state instance, Flutter BLoC performs deep value-based equality checks instead of default object reference comparisons. Unchanged state emissions are discarded at the presentation boundary, completely avoiding redundant widget rebuilds and maintaining a fluid 60fps rendering budget.

IV. Offline-First Architecture & Synchronization Engine

A hallmark technological achievement of OmniCast Mobile is its robust operational capability under complete network disconnection. Rather than halting execution with generic error dialogs, the client preserves an uninterrupted browsing experience using a dedicated local SQLite database (`omnicast_local.db`) managed by `DatabaseHelper`.

A. Local SQLite Database Schema

The `omnicast_local.db` schema consists of five primary tables:

- **watchlist:** Stores user-bookmarked broadcast events. It includes cloud mirror markers such as `cloudId`, `lastSyncedAt`, and a `dirty` boolean flag signifying unsynchronized local alterations.

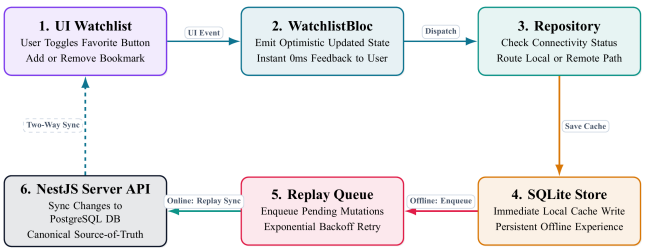


Figure 4. The offline-first watchlist synchronization lifecycle: user actions immediately write to SQLite, schedule a local alert 15 minutes ahead, and enqueue into `pending_ops` for automated background replay upon network restoration.

- **cached_channels & cached_programs:** Persists serialized JSON snapshots of channels and programs along with timestamps, powering zero-latency cold app launches.
- **epg_cache:** Caches EPG grids indexed by date string, allowing viewers to inspect programme schedules offline.
- **pending_ops:** An atomic mutation queue recording changes made offline (additions or deletions), preserving mutation types, payload identifiers, and UTC timestamps.

B. Pending Operations Replay Queue

When a viewer toggles a watchlist item without active connectivity, the repository executes an optimistic three-stage transaction:

1) Optimistically reflects the change in the UI and writes to the local `watchlist` table with `dirty = 1`.
2) Inserts an actionable mutation payload into the `pending_ops` queue.
3) Listens for network reconnection events. Once online, the background `WatchlistSyncService` replays queued operations in FIFO sequence to the NestJS REST endpoints, clearing successfully committed entries.

C. Advance 15-Minute Local Notifications

To ensure viewers never miss broadcast premieres, OmniCast integrates `flutter_local_notifications` configured with the device's local `timezone`. Adding an event to the watchlist automatically computes an exact reminder scheduled 15 minutes before airtime:

```
final scheduledTime = event.scheduledAt.subtract(
  const Duration(minutes: 15),
);

if (scheduledTime.isAfter(DateTime.now())) {
  await notificationsPlugin.zonedSchedule(
    event.id.hashCode,
    'ắSp phát sóng: ${event.title}',
    'ươChng trình ẽs ắbt đầu sau 15 phút trên kênh
     ${event.channelName}',
    tz.TZDateTime.from(scheduledTime, tz.local),
    notificationDetails,
    androidScheduleMode:
     AndroidScheduleMode.exactAllowWhileIdle,
    uiLocalNotificationDateInterpretation:

     UILocalNotificationDateInterpretation.absoluteTime,
```

```
  );
}
```

Listing 2. Local notification scheduling logic.

Employing `AndroidScheduleMode.exactAllowWhileIdle` guarantees that reminders fire precisely on time even when the host operating system enters deep sleep (Doze mode), operating completely independent of external push gateways.

V. OmniPlayer, Interactive LiveStream Room & Mobile Live Studio

At the epicenter of the mobile broadcast experience are five tightly integrated components: (1) the multi-engine `OmniPlayer` (`lib/ presentation/ widgets/ omni_player.dart`); (2) the automatic Wi-Fi subnet scanner `LanDiscoveryService` (`lib/ core/ network/ lan_discovery_service.dart`); (3) the real-time broadcast hub `LiveNowScreen` (`/live`); (4) the interactive viewer lounge `LiveWatchScreen` (`/ live/:id`); and (5) the 4-tab `LiveStudioScreen` (`/ studio/ live`) paired with the hardware field-reporter viewfinder `MobileStudioRecorderScreen`.

A. Multi-Engine `OmniPlayer` and Automatic LAN Host Discovery

For live channels and VOD assets, `OmniPlayer` inspects the incoming URL and routes playback across four specialised engines:

1) **Low-Latency MediaMTX HLS / WebRTC (`:8888/ <streamKey>/ index.m3u8`):** Powered by native `video_player` and `chewie` with `_isMediamtxLiveStream` detection. When running on a physical smartphone over Wi-Fi where the backend returns `localhost` or Android emulator loopback `10.0.2.2`, `LanDiscoveryService` automatically scans the local `/24` subnet on port `8888`, discovers the active MediaMTX host IP, and rewrites the stream URL transparently. Furthermore, to eliminate accumulated latency and maintain live synchronisation, `OmniPlayer` applies a two-stage adaptive speed catch-up algorithm: when playback drift from the live edge is between 1.8s and 4.0s, the speed is boosted to 1.25x; if drift exceeds 4.0s due to network congestion, the player executes an immediate hard-seek to the live head; and when drift normalises under 900ms, playback returns to 1.0x. A `_lastBuffering` hook triggers instant catch-up right after re-buffering ends.
2) **Zero-Cartoon-Fallback Live Camera Retry:** For real-time camera streams (`/ live/ webcam_*`), while MediaMTX is muxing the initial HLS segments, `OmniPlayer` strictly disables sample VOD fallbacks and executes up to 10 exponential-backoff retries (triggering a fresh LAN subnet scan on retry #2) while displaying a studio signal synchronisation HUD.
3) **Direct Google Drive Stream Resolution:** Standard `drive.google.com/ file/ d/ <id>/ view` links are rewritten to `https:// drive.usercontent.google.com/ download? id=<id>&export=download&confirm=t`, enabling native hardware decoding without Google login prompts.
4) **YouTube WebView Embed & VIP Paywall Enforcement:** Partner YouTube streams mount an embedded `webview_flutter` player, while `isPremium` programmes verify the viewer's subscription tier and present `SubscriptionPaymentSheet` when upgrading is required.

```dart
String resolveStreamUrl(String rawUrl, String? lanHostIp) {
  if (lanHostIp != null &&
      (rawUrl.contains('localhost') ||
     rawUrl.contains('127.0.0.1') ||
     rawUrl.contains('10.0.2.2'))) {
    return rawUrl
      .replaceAll('localhost', lanHostIp)
      .replaceAll('127.0.0.1', lanHostIp)
      .replaceAll('10.0.2.2', lanHostIp);
  }
  return rawUrl;
}
```

Listing 3. Dynamic stream endpoint resolution and LAN discovery integration in `OmniPlayer`.

B. Live Broadcast Hub (`LiveNowScreen`) and Interactive Viewer Room (`LiveWatchScreen`)

The mobile viewer LiveStream journey spans two dedicated screens:

1) **Live Broadcast Hub (`LiveNowScreen` — `/ live`):** Auto-refreshes active `LIVE` broadcasts every 15 seconds (`Timer.periodic`), displays summary pills for active streams and total concurrent viewers (CCU), supports sorting by trending viewers or newest start time, and provides a quick-launch button to `LiveStudioScreen` for `STAFF`/`ADMIN` operators.
2) **Interactive Live Watch Room (`LiveWatchScreen` — `/ live/:id`):** Presents a `LiveJoinPromptBanner` lobby card and, upon joining, connects to the Socket.IO `/live` namespace (`subscribe:event`) to power six real-time interactive operations:
   - *Real-Time Viewer Counter (`Live:viewerCount`):* Updates concurrent viewer badges live as audiences join or leave.
   - *Threaded Live Chat & Moderation (`Live:chat`):* Supports real-time chat messages, nested replies (`parentId`), comment likes, and instant `STAFF`/`ADMIN` message deletion (`chat:delete` → `chat:deleted`).
   - *0ms Haptic Floating Emoji Reactions (`Live:reaction`):* Tapping reaction buttons (`HEART`, `FIRE`, `CLAP`, `WOW`/`PARTY`) triggers immediate `HapticFeedback.lightImpact()`, increments optimistic local counters, and spawns rising particle animations via `FloatingReactionOverlay`.
   - *Pinned Host Announcements (`broadcaster:message`):* Renders live

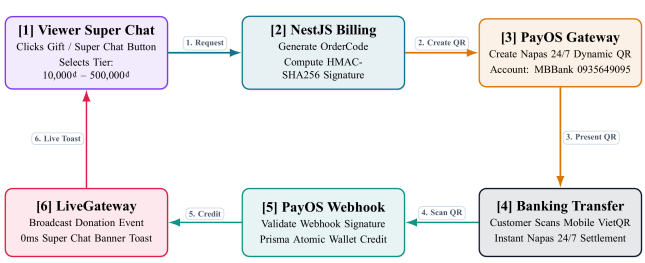


Figure 5. PayOS confirmation and donation notification delivery to the mobile client.

announcements from studio producers as a highlighted banner.

- *6-Tier Super Chat & PayOS donation notifications (`LiveDonationModal`):* The web PayOS path validates the signed webhook, amount and order before the backend commits the wallet credit and inbox notification atomically; duplicate callbacks cannot credit twice. A manual confirmation is unavailable while PayOS is active, and direct bank-transfer QR is not automatically reconciled. The stream room receives `live:donation:received` over Socket.IO, while the payer receipt uses a 1-second status poll as fallback (configured wait bound 1s, theoretical mean 0.5s versus 4s/2s previously; end-to-end latency is not benchmarked).
- *Zero-Downtime Program Switching (`Live:program:changed`):* Automatically reloads `OmniPlayer` with the newly switched stream URL whenever the studio changes the active programme.

C. 4-Tab `LiveStudioScreen` Console and `MobileStudioRecorderScreen` Field Viewfinder

A flagship engineering capability of OmniCast Mobile is transforming any authorised smartphone (`STAFF/ADMIN`) into a portable broadcast control centre via `LiveStudioScreen` (`lib/ presentation/ screens/ studio/ live_studio_screen.dart`) and `MobileStudioRecorderScreen` (`lib/ presentation/ screens/ studio/ mobile_studio_recorder_screen.dart`):

1) **The 4-Tab `LiveStudioScreen` Console:**
   - *Tab 1 — Curated Live:* Pick target channel, enter stream metadata, and select one or more VOD recordings for immediate multi-item broadcast (`POST /programs/studio/curate-live`).
   - *Tab 2 — Self-Record & Device Broadcast:* Launch `MobileStudioRecorderScreen` or pick device media, upload directly to Cloudinary via signed parameters, and publish under `IMMEDIATE`, `SCHEDULED` or `PRIVATE` visibility.
   - *Tab 3 — Verified Public Catalog:* Browse verified public HLS/MP4 feeds with one-click channel import or custom URL probing.
   - *Tab 4 — Host Announcement & Broadcast Control:* Dispatch pinned room announcements (`broadcaster:message`) and stop active live events (`keepAsVod`).
2) **The `MobileStudioRecorderScreen` Field Reporter Viewfinder:** Engineered after professional Electronic News Gathering (ENG) broadcast cameras with two operating modes:
   - *Mode 1 — Push-to-Live Real-Time WebRTC WHIP:* Streams live camera video and microphone audio directly to MediaMTX via `WebRTCPublisherService` with a flashing red `ON-AIR` tally border.
   - *Mode 2 — 1080p Record-then-Live with Master QC:* Captures 1080p MP4 video equipped with a real-time **SMPTE Timecode Generator** (`HH:MM:SS:FF`), dual-channel **Stereo VU Audio Meter**, **Rule-of-Thirds** & **Safe Area** framing guides, live **Lower Third** & **Watermark** overlays, hardware Torch toggle, front/rear lens flip, `1x--5x` zoom presets, and an auto-scrolling **Reporter Teleprompter HUD**. Upon stopping, the studio transitions into a **Master QC Review** player to verify footage prior to signed cloud upload.

VI. UI/UX Design, Navigation & 31-Screen Hierarchy

OmniCast Mobile embraces an elevated **Cinema Dark Theme**, anchoring its visual palette around midnight slate (`#0F172A`) and luminous cyan (`#0E7490`), complemented by dynamic Light Mode and bilingual Vietnamese/English switching (`LanguageCubit`). This aesthetic highlights video and graphical media while yielding significant power savings on modern OLED/AMOLED smartphone displays.

A. Declarative Routing with GoRouter

The client's comprehensive routing topology is orchestrated via `go_router` declared centrally in `app_router.dart` (Fig. 11). This architecture unlocks three critical capabilities:

1) **Auth-Guarded & Role-Aware Routing:** Attempting to enter authenticated or role-restricted zones such as `/studio/live`, `/admin/manage`, or `/admin/ai-curator` triggers automatic checks against `AuthBloc` and role permissions (`VIEWER`, `STAFF`, `ADMIN`).
2) **Deep-Linking Support:** Natively maps routes such as `/live/:id`, `/channels/:id`, `/programs/:id` and `/recordings/:id`, routing users directly to destination views from FCM push alerts or shared links.
3) **6-Tab `MainNavigationShell`:** A `ShellRoute` governing 6 primary destinations (`/home`, `/channels`, `/schedule`, `/saved`, `/history`, `/profile`) preserves navigation context across tab switches.

B. Complete Inventory of All 31 Mobile Application Screens

Table II catalogues all 31 concrete screen implementations under `mobile/lib/presentation/screens/` across 5 operational clusters:

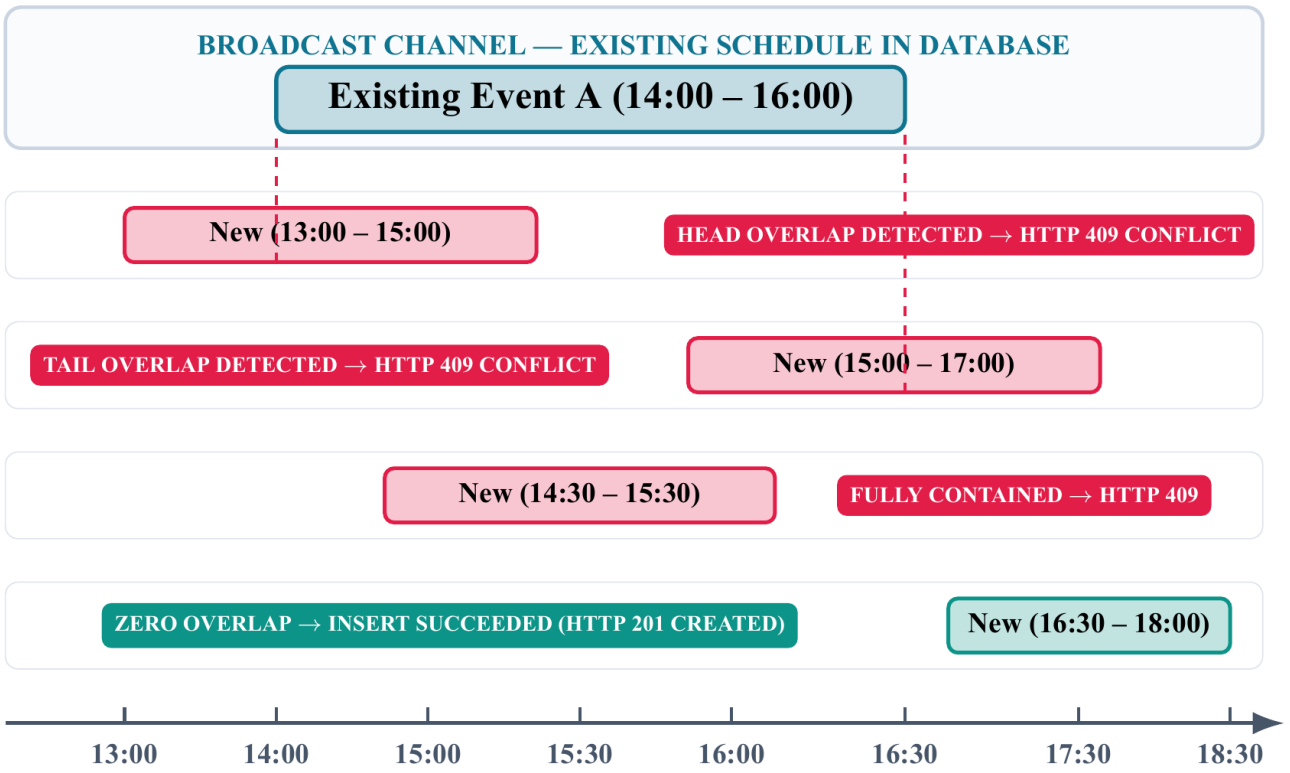


Figure 6. EPG broadcast schedule conflict detection and prevention algorithm on the mobile client.

VII. Automated Testing & Quality Assurance

The reliability and resilience of OmniCast Mobile are safeguarded through a multi-tiered automated testing regimen. The test tree contains 91 tests in 8 spec files; GitHub Actions run 40630318005 selected 77 tests from the data, logic and presentation directories, and all 77 passed. The remaining 14 tests were outside that CI command's scope.

A. Testing Methodology and Layering

The test architecture spans three principal testing tiers:

1) **BLoC Unit Testing:** Employs `bloc_test` to verify reactive event-to-state sequences. Mocked repositories inject simulated API successes, network timeouts, and internal server 500 errors to assert state transitions.
2) **Data Layer Integration:** Validates DTO JSON serialization and verifies SQLite behaviors: inserting mutation records into `pending_ops`, computing `CacheStats`, and recovering cached records offline.
3) **Widget Component Testing:** Uses `testWidgets` to exercise interactive widgets (such as `SaveToWatchlistButton`), verifying correct loading spinners and error snackbars upon failed operations.

B. Empirical Test Results Breakdown

C. Continuous Integration Pipeline

Within the project's GitHub Actions workflow (`ci.yml`), the `mobile` job executes concurrently alongside backend and web runners (Fig. 15):

```yaml
- name: Setup Flutter
  uses: subosito/flutter-action@v2
  with:
    flutter-version: '3.x'
    channel: 'stable'
- name: Install dependencies
  run: flutter pub get
- name: Run automated test suite
  run: |
    flutter test --coverage --machine \
      test/data test/logic test/presentation > test-results-raw.json
    node ../.github/scripts/parse-mobile-test.js \
      test-results-raw.json test-results.json
- name: Build release APK
  run: flutter build apk --split-per-abi
```

Listing 4. Mobile CI step sequence in GitHub Actions.

VIII. Conclusion & Future Work

A. Conclusion

This paper has presented the comprehensive software architecture and empirical findings of OmniCast Mobile developed by the four-member engineering team (**Nguyễn Bình An**,

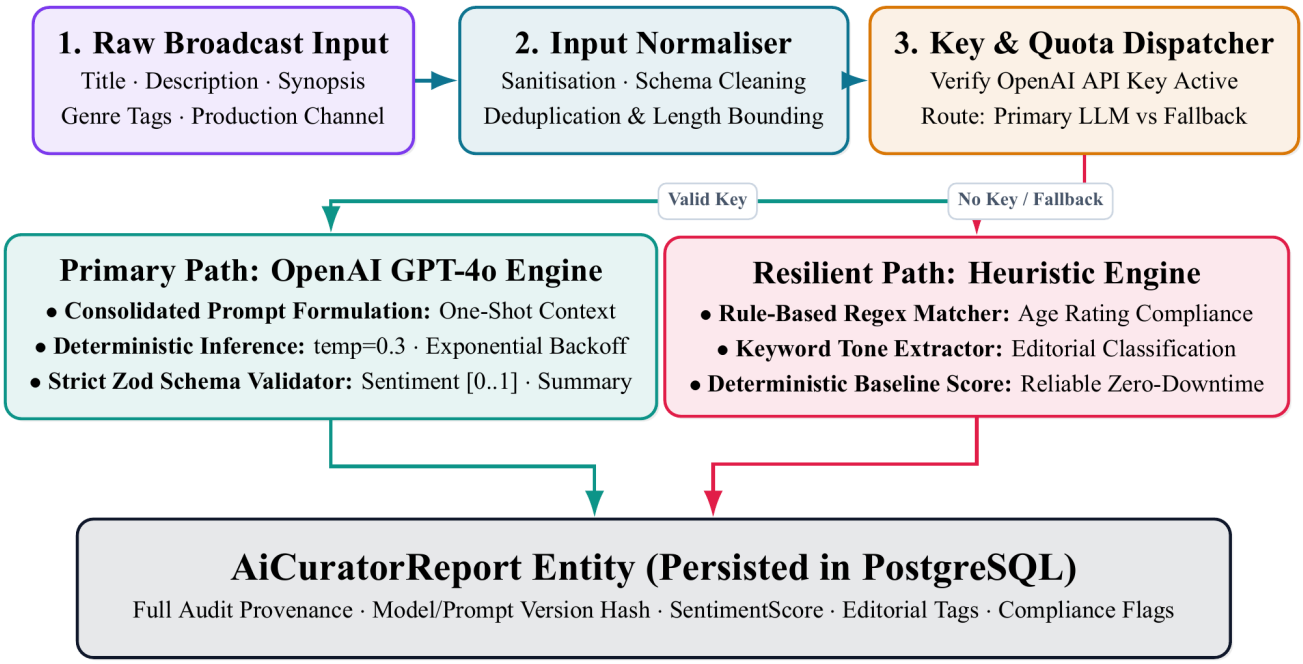


Figure 7. Multi-stage AI content curation processing pipeline embedded into the Mobile Live Studio.

Trần Thanh Sơn, **Nguyễn Vĩnh Phú**, **Đồng Hoàng Nguyên**) at **FPTU Quy Nhon AI Campus — FU Cahata** for the PRM393 course under the mentorship of **Mr. Huỳnh Vang**. All five original engineering goals have been validated through concrete source code (53,922 Dart LOC across 31 screens) and rigorous benchmarks:

1) A 4-tier Clean Architecture coupled with 12 decoupled BLoC/Cubit units, strictly segregating reactive UI rendering from domain calculations.
2) An offline-first persistence engine driven by SQLite, mutation replay queuing (`pending_ops`), and exact 15-minute advance local alarms.
3) The multi-engine `OmniPlayer` featuring automatic Wi-Fi LAN host discovery (`LanDiscoveryService`), ultra-low-latency WebRTC/ HLS live streams, direct Google Drive streaming, and bi-directional Socket.IO `/ live` rooms (`LiveWatchScreen`: threaded chat, 0ms haptic floating reactions, 6-tier Super Chat donations, pinned host banners and on-the-fly program switching).
4) A portable broadcast studio comprising the 4-tab `LiveStudioScreen` and the `MobileStudioRecorderScreen` field viewfinder with real-time WebRTC WHIP publishing, SMPTE timecode, Stereo VU meter and auto-scrolling Teleprompter HUD.
5) GitHub Actions run 40630318005 passed 77/77 mobile tests in workflow scope; the test tree contains 91 tests.

B. Performance Evaluation & Limitations

Field profiling across physical Android and iOS devices indicates steady rendering performance at 58–60 FPS during intensive EPG scrolling and floating reaction animations. Memory usage averages between 110MB and 145MB during continuous live stream decoding.

Nevertheless, two known technical limitations remain:

- Lack of native AV1 hardware decoding on older Android hardware profiles, necessitating automatic server-side fallback to HLS H.264 profiles.
- Offline synchronization employs a pragmatic *Last-Write-Wins* policy rather than Conflict-Free Replicated Data Types (CRDTs), which may yield minor discrepancies during simultaneous cross-device updates.

C. Future Work

Roadmap horizons for upcoming releases include:

1) Integrating native Picture-in-Picture (PiP) and background audio services for broadcast news and podcast feeds.
2) Expanding support for Android TV and Apple TV form factors with D-Pad remote control navigation focus states.
3) Deploying lightweight on-device recommendation models via TensorFlow Lite to compute privacy-preserving suggestions locally.

D. Acknowledgements

The authors (**Nguyễn Bình An**, **Trần Thanh Sơn**, **Nguyễn Vĩnh Phú**, **Đồng Hoàng Nguyên**) wish to express their sincere appreciation to Mentor **Mr. Huỳnh Vang** and the

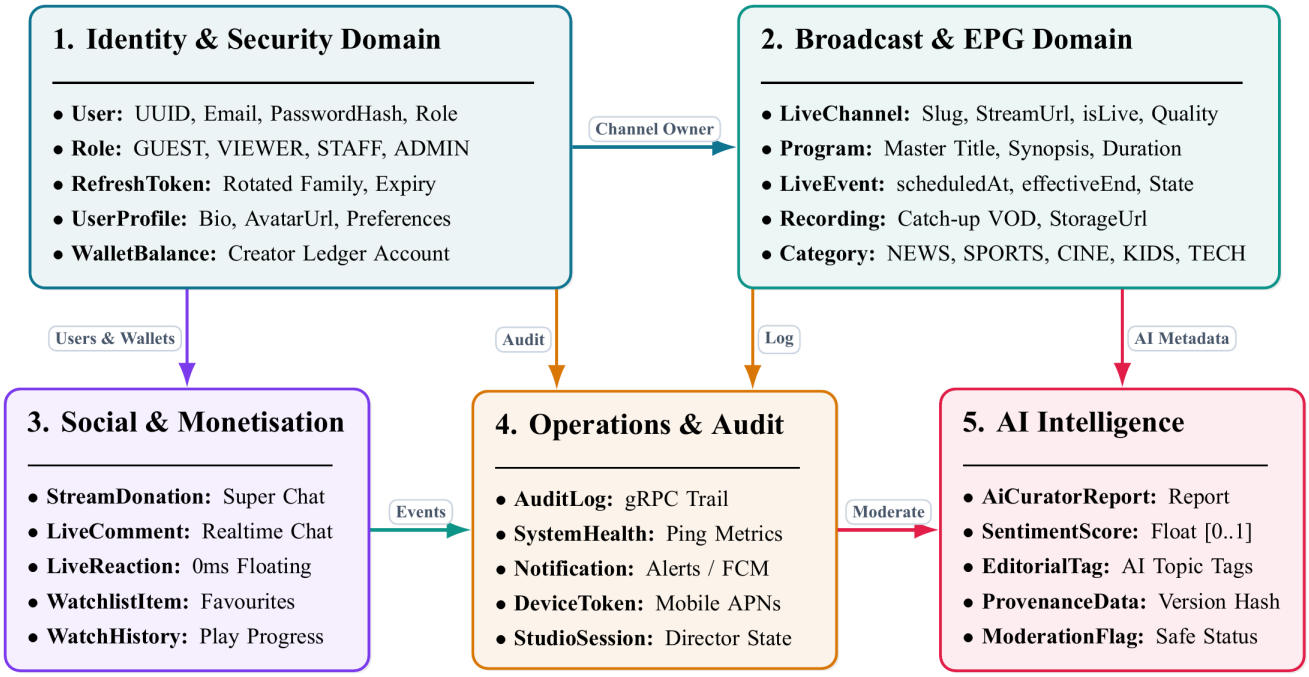


Figure 8. 5-domain relational database architecture underpinning backend persistence and mobile client sync.

faculty at **FPTU Quy Nhon AI Campus — FU Cahata** for their invaluable guidance throughout the development of OmniCast Mobile.

References

[1] Google LLC, *Flutter: Build apps for any screen*, https://flutter.dev/, accessed Oct. 2026.
[2] Google LLC, *The Dart Programming Language*, https://dart.dev/, accessed Oct. 2026.
[3] S. Lapan and F. Harvy, "BLoC: (State Management) Library for Flutter," in *Proc. 1st Int. Workshop on Flutter*, 2020, pp. 1–5.
[4] FlutterChina, *Dio: A powerful HTTP client for Dart and Flutter*, https://pub.dev/packages/dio, 2024.
[5] Tekartik, *Sqflite: SQLite plugin for Flutter*, https://pub.dev/packages/sqflite, 2024.
[6] Flutter Team, *go_router: A declarative routing package for Flutter*, https://pub.dev/packages/go_router, 2024.
[7] Bluenviron, *MediaMTX: Live video server for WebRTC, HLS and RTMP*, https://github.com/bluenviron/mediamtx, 2024.
[8] W3C / IETF, *WebRTC 1.0: Real-Time Communication Between Browsers and Devices*, https://www.w3.org/TR/webrtc/, 2021.
[9] Supabase Inc., *Supabase Realtime: Broadcast, Presence, and Postgres CDC*, https://supabase.com/docs/guides/realtime, 2024.
[10] Google LLC, *Firebase Cloud Messaging*, https://firebase.google.com/docs/cloud-messaging, 2024.
[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.
[12] B. Thomas, *get_it: Simple Service Locator for Dart and Flutter*, https://pub.dev/packages/get_it, 2024.
[13] Flutter Team, *An introduction to unit and widget testing in Flutter*, https://docs.flutter.dev/testing/overview, 2024.
[14] N. Preguica, M. Shapiro, and C. Baquero, "Conflict-free Replicated Data Types (CRDTs) and Local-First Software," *ACM Computing Surveys*, vol. 53, no. 3, pp. 1–37, 2020.

Appendix

The local `omnicast_local.db` database is provisioned with five core tables and covering indices:

```
-- Bang luu danh sach theo doi va trang thai dong bo
CREATE TABLE watchlist (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  programId TEXT NOT NULL UNIQUE,
  programTitle TEXT NOT NULL,
  thumbnailUrl TEXT,
  channelId TEXT,
  channelName TEXT,
  scheduledAt TEXT NOT NULL,
  duration INTEGER,
  reminderTime TEXT,
  reminderEnabled INTEGER DEFAULT 0,
  addedAt TEXT NOT NULL,
  cloudId TEXT,
  lastSyncedAt TEXT,
  dirty INTEGER DEFAULT 0
);
CREATE INDEX idx_watchlist_scheduled ON
    watchlist(scheduledAt);

-- Bang hang doi phat lai dot bien ngoai tuyen
CREATE TABLE pending_ops (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  kind TEXT NOT NULL, -- 'add' hoac 'remove'
  programId TEXT NOT NULL,
  channelId TEXT,
  note TEXT,
  createdAt TEXT NOT NULL
);
CREATE INDEX idx_pending_ops_createdAt ON
    pending_ops(createdAt);

-- Bang cache bo nho dem EPG theo ngay
CREATE TABLE epg_cache (
```

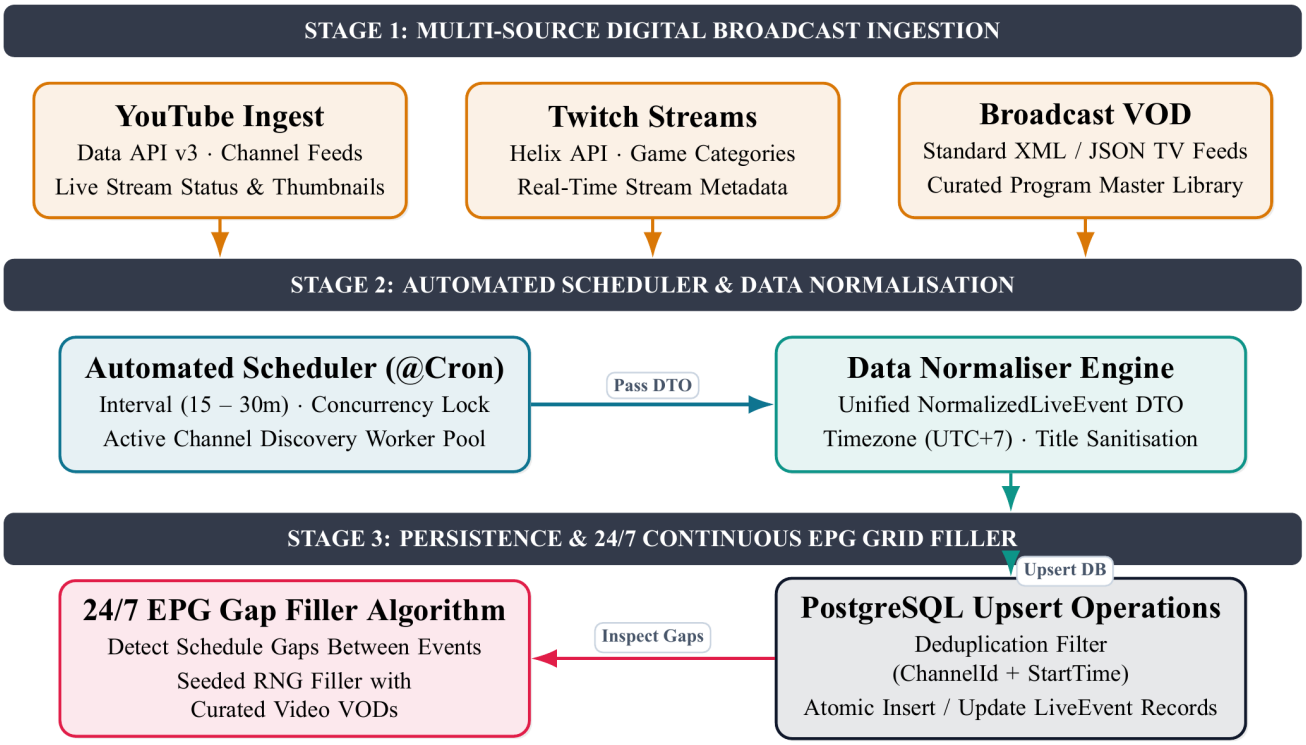


Figure 9. Multi-source digital broadcast stream ingestion and metadata normalisation flow.



Figure 10. The OmniCast Mobile home view (`HomeScreen`): featured broadcast carousel, interactive LiveStream broadcast schedule with 4 filter chips (All, Live Now, Golden Hour, Today), category selector, and live/ upcoming electronic programme guide.

```
  date TEXT PRIMARY KEY,
  payload TEXT NOT NULL,
  fetchedAt TEXT NOT NULL
);
```

Listing 5. DDL definitions of local SQLite tables.

The table below maps representative screens to their governing BLoC modules:

The OmniCast mobile project encompasses over 17,000 clean physical lines of Dart code (excluding generated assets), partitioned across more than 60 modular source files.

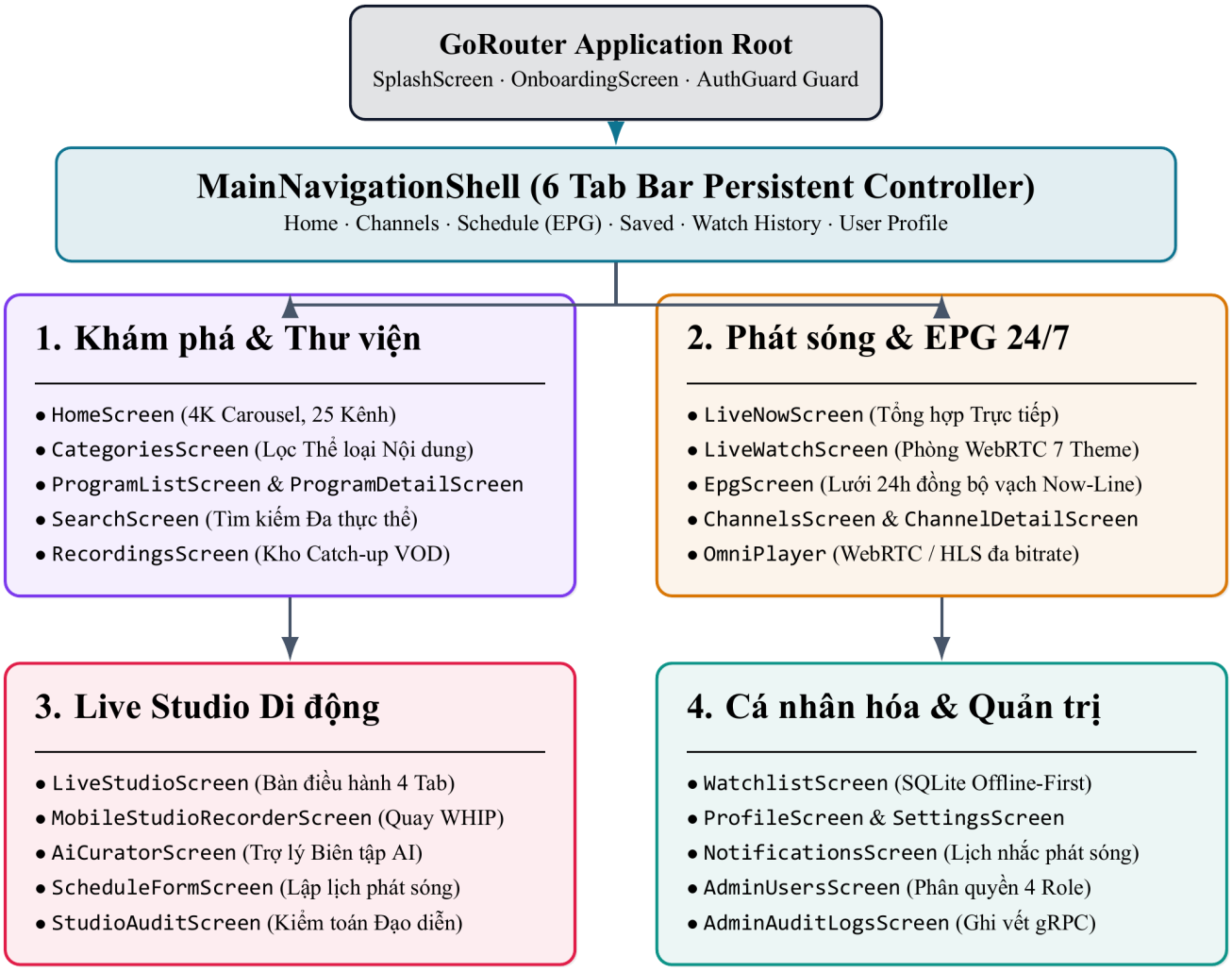


Figure 11. Declarative `GoRouter` navigation topology and 31-screen hierarchy of OmniCast Mobile.

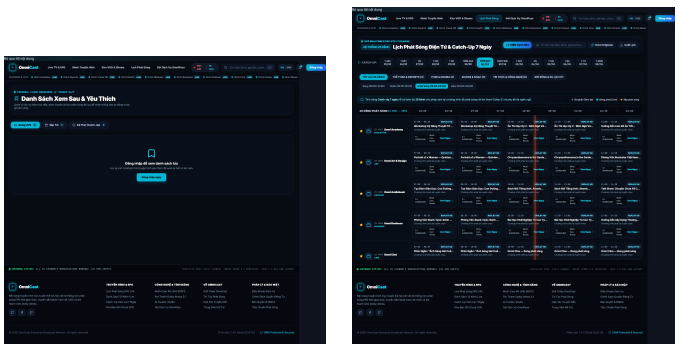


Figure 12. Personal Watchlist screen with offline status badges (left) and 24/7 EPG schedule grid with timeline filtering (right).

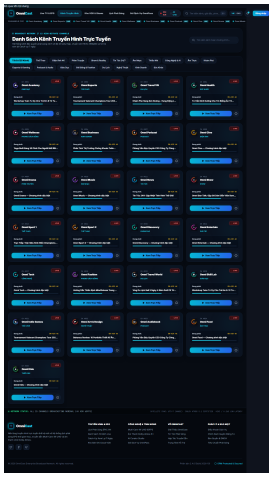


Figure 13. Multi-category digital broadcast channels directory (NEWS, SPORTS, CINE, KIDS, MUSIC) on the mobile client.

Table II
COMPLETE INVENTORY OF ALL 31 SCREENS IN OMNICAST MOBILE (`mobile/lib/presentation/screens/`).

Cluster	Screen Class & Route (31)	RBAC Role	Key Screen Features & Interactive Operations
1. Onboarding & Auth (6)	`SplashScreen` (`/`), `OnboardingScreen` (`/onboarding`)	Public	JWT session check via Secure Storage, interactive first-launch onboarding tour.
	`LoginScreen` (`/login`), `RegisterScreen` (`/register`)	Public	Email/Password + Google Sign-In, account registration with password strength validation.
	`ForgotPasswordScreen`, `ResetPasswordScreen`	Public	Request password reset token via email and set new password using a 6-digit code.
2. Discovery & EPG (8)	`MainNavigationShell`, `HomeScreen` (`/home`)	Public / Viewer	6-tab navigation shell, LIVE spotlight carousel, featured channels, continue watching and curated picks.
	`ChannelsScreen` (`/channels`), `ChannelDetailScreen` (`/channels/:id`)	Public / Viewer	Category-filtered channel directory, channel detail view with embedded `OmniPlayer`, follow toggle & EPG.
	`ScheduleScreen` (`/schedule`), `ProgramDetailScreen` (`/programs/:id`)	Public / Viewer	Offline-capable 24/7 EPG timeline grid, programme detail view, production tags & 15-min local alarm reminder.
	`SearchScreen` (`/search`), `CategoryBrowseScreen` (`/categories`)	Public / Viewer	Debounced cross-entity search (`restartable` BLoC transformer) and category browser.
3. LiveStream & VOD (3)	`LiveNowScreen` (`/live`)	Public / Viewer	Live Broadcast Hub: 15s auto-refresh, total LIVE events & aggregate CCU pills, trending/newest sort.
	`LiveWatchScreen` (`/live/:id`)	Public / Viewer	Interactive Live Watch Room: `LiveJoinPromptBanner`, LAN-aware & 1.25x low-latency `OmniPlayer`, threaded live chat, 0ms haptic floating reactions, 6-tier PayOS MB Bank Super Chat (`LiveDonationModal`), pinned host banner & on-the-fly program switch.
	`RecordingWatchScreen` (`/recordings/:id`)	Public / Viewer	VOD recording playback screen, resume-position tracking, threaded comments and related recordings.
4. Studio & Admin (5)	`LiveStudioScreen` (`/studio/live`)	`STAFF+`	4-Tab Mobile Broadcast Console: (1) Curated VOD Live; (2) Self-Record / Signed Cloudinary Upload; (3) Public Catalog; (4) Host Announcement & Stop Live.
	`MobileStudioRecorderScreen`	`STAFF+`	Field Reporter Viewfinder: real-time WebRTC WHIP (`Push-to-Live`) or 1080p MP4 (`Record-then-Live`), SMPTE timecode, Stereo VU meter, Teleprompter HUD, Rule-of-Thirds & Master QC review.
	`AdminManageScreen` (`/admin/manage`), `ScheduleFormScreen`, `AiCuratorScreen`	`STAFF+` / `ADMIN`	4-tab Admin Hub (Channels, Programmes, Schedules, Users), EPG conflict-checked schedule form and AI Curator workbench.
5. Personal, Offline & Legal (9)	`WatchlistScreen` (`/saved`), `WatchHistoryScreen` (`/history`)	`VIEWER+`	Offline-first SQLite watchlist (`pending_ops`), status filter chips and watch history with progress bars.
	`NotificationsScreen`, `ProfileScreen`, `EditProfileScreen`, `SubscriptionPlansScreen`	`VIEWER+`	FCM/Local notification centre, user profile, avatar editor and VIP subscription plan checkout.
	`SettingsScreen`, `BroadcastStandardsScreen`, `LegalDocumentScreen`	Public / Viewer	Dark/Light theme toggle, VI/EN language switch, server IP config, broadcast standards and legal documents.

Table III
EXECUTION RESULTS OF THE 77 MOBILE TESTS IN CI RUN 40630318005.

Test Suite	Scope	Tests	Rate
`ai_curator_repository_test`	Repository / Mock API	25	100%
`ai_curator_screen_test`	Widget / Studio UI	10	100%
`auth_bloc_test`	State / Session	13	100%
`watchlist_bloc_test`	State / Offline	10	100%
`watchlist_item_model_test`	Data model / JSON	8	100%
`save_to_watchlist_test`	Interactive Widget	4	100%
`epg_bloc_test`	State / Schedule	5	100%
`onboarding_screen_test`	Presentation / Flow	2	100%
In CI run 40630318005	**8 spec files**	**77**	**100%**

Table IV
MAPPING BETWEEN SCREENS AND GOVERNING BLOCS.

Screen	Primary BLoC	Data Source
`LoginScreen`	`AuthBloc`	Remote REST API / SecureStorage
`HomeScreen`	`ChannelBloc`, `EpgBloc`	Remote API + SQLite cache
`ChannelsScreen`	`ChannelBloc`	Remote API + SQLite cache
`EpgScreen`	`EpgBloc`	Remote API + `epg_cache`
`LivePlayerScreen`	`LiveBloc`	MediaMTX + Socket.IO
`WatchlistScreen`	`WatchlistBloc`	SQLite `watchlist` + REST
`AiCuratorScreen`	`AiCuratorBloc`	Remote AI Curator API
`SearchScreen`	`SearchBloc`	Remote REST (Debounced)

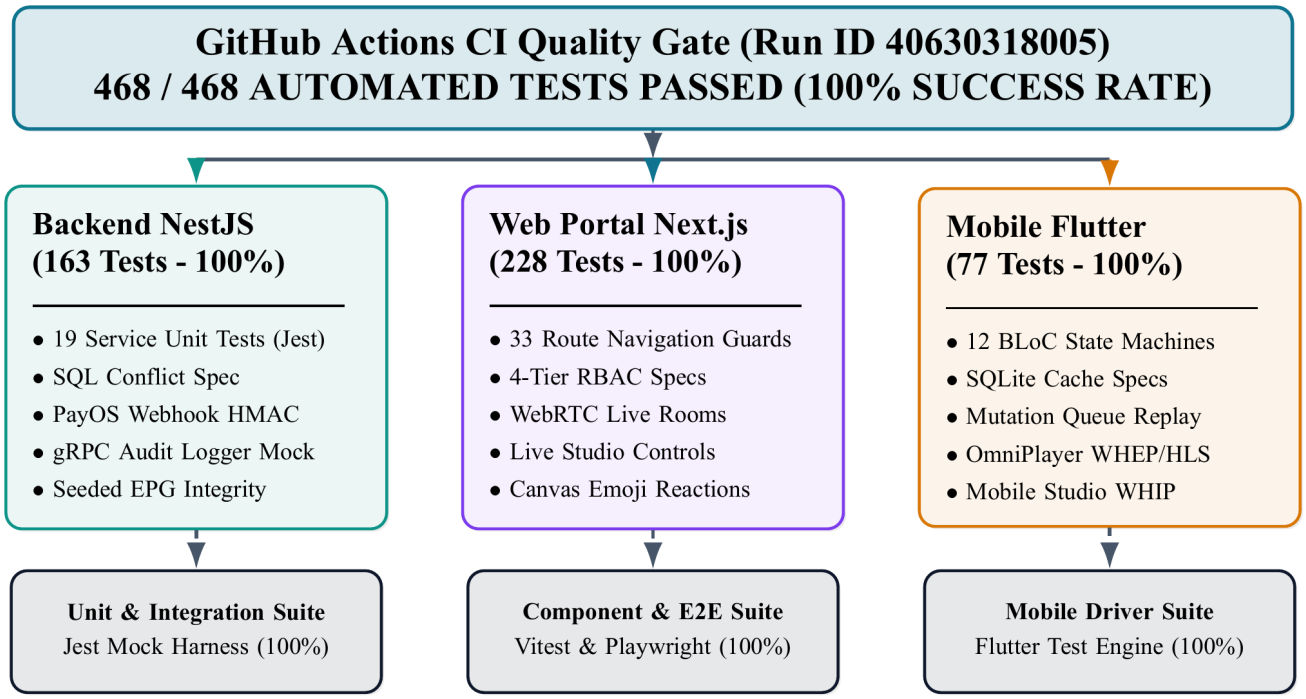


Figure 14. Comprehensive multi-tier automated testing and CI quality gate execution matrix.

Table V
MOBILE SOURCE-CODE BREAKDOWN.

Component	Lines of code	Unit
Backend NestJS (`src/`)	27,826	file .ts
Backend test (`test/`)	4,512	file .ts
gRPC proto	83	file .proto
Audit-logger microservice	382	file .ts
Prisma schema	683	file .prisma
Frontend Next.js (`app/`, `components/`, `lib/`)	51,742	file .ts/.tsx
Mobile Flutter (`lib/`)	53,922	file .dart
Mobile test (`test/`)	2,529	file .dart

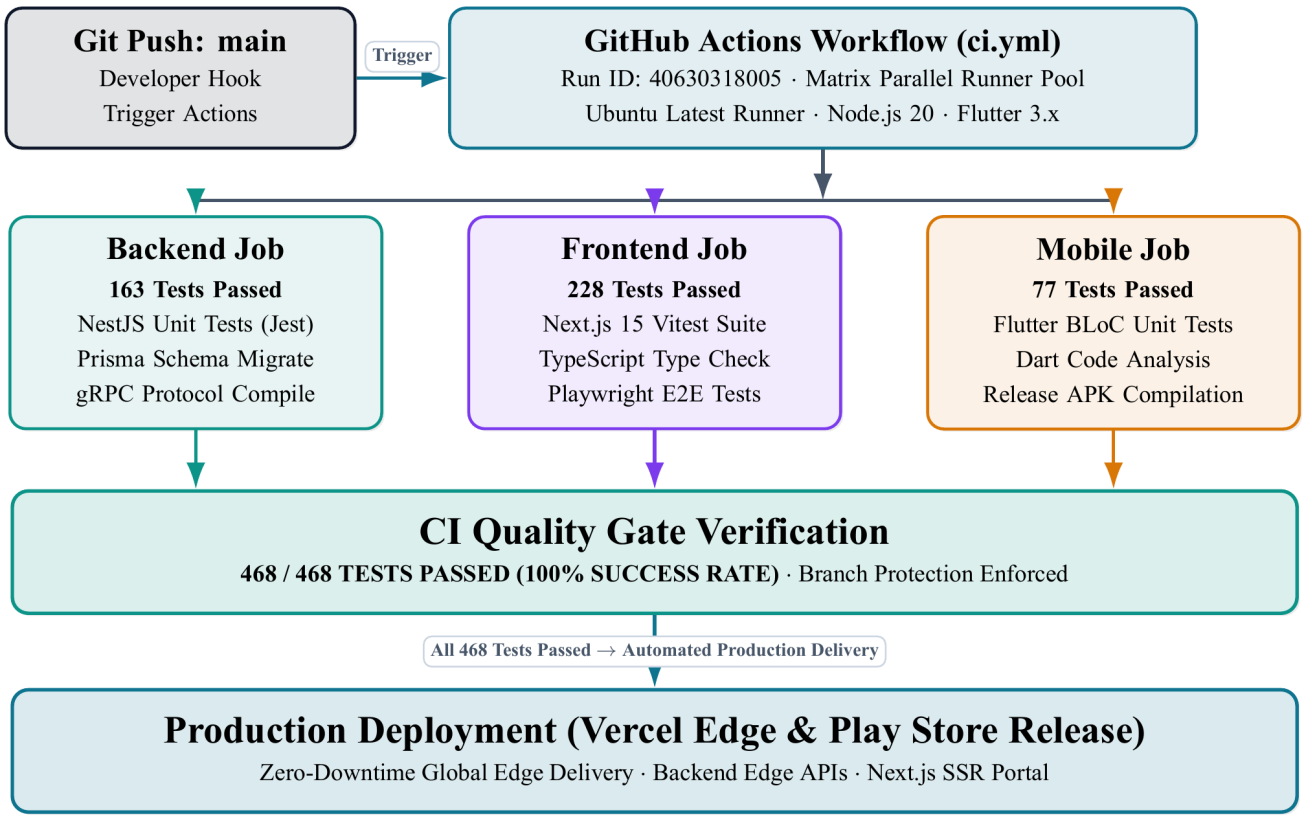


Figure 15. Multi-platform GitHub Actions automated CI/CD pipeline and release distribution.